ALTO GUITAR FX

RANCANGAN APLIKASI ANDROID
PROFESSIONAL REAL-TIME GUITAR MULTI EFFECTS

Dokumen rancangan teknis • Arsitektur • Fitur • Struktur proyek • Roadmap pembangunan

ALTOMEDIA

Versi dokumen: 1.0
Status: Rancangan awal untuk memulai pengembangan. Dokumen ini belum merupakan aplikasi jadi atau kode

sumber lengkap.

Daftar isi

- 1. Ringkasan produk dan tujuan
- 2. Kebutuhan fungsional dan batasan
- 3. Arsitektur sistem dan alur audio
- 4. Teknologi yang digunakan
- 5. Struktur folder proyek Android Studio
- 6. Rancangan antarmuka dan navigasi
- 7. Katalog efek dan parameter
- 8. Preset awal
- 9. Spesifikasi audio dan performa
- 10. Izin Android, perangkat, dan kompatibilitas
- 11. Tahapan pembangunan per BAB
- 12. Rencana pengujian dan kriteria selesai
- 13. Risiko teknis dan mitigasi
- 14. Checklist sebelum rilis APK

1. Ringkasan produk dan tujuan

Nama: ALTO GUITAR FX
Pengembang/brand: ALTOMEDIA
Package name usulan: com.altomedia.altoguitarfx
Platform: Android
Jenis: Multi Effects Profesional
Mode: Pemrosesan suara real-time.

ALTO GUITAR FX dirancang sebagai prosesor efek gitar digital. Pengguna dapat memasukkan suara gitar melalui mikrofon internal HP atau USB Audio Interface, mengaktifkan beberapa efek secara berurutan, mengatur parameter setiap efek, lalu mendengarkan hasilnya melalui headphone atau perangkat output yang kompatibel.

Tujuan utama adalah membangun jalur audio yang benar-benar memproses sinyal secara langsung. Tombol dan knob pada antarmuka harus terhubung ke parameter DSP yang bekerja pada audio, bukan sekadar tampilan visual.

2. Kebutuhan fungsional dan batasan

Fitur inti

- Memilih sumber input: mikrofon internal atau USB Audio Interface.
- Mendeteksi dan menampilkan perangkat audio yang tersedia.
- Memilih output yang didukung Android/perangkat USB.
- Memproses audio real-time dengan rangkaian efek yang dapat diaktifkan/nonaktifkan.
- Mengatur parameter efek melalui knob, slider, dan tombol bypass.
- Menyediakan preset bawaan serta menyimpan preset buatan pengguna.

- Menyediakan master input/output level dan indikator level audio.
- Menangani izin mikrofon, koneksi USB, perubahan perangkat, dan audio interruption.

Batasan dan asumsi

- Mikrofon internal HP menangkap suara gitar dari udara; kualitasnya dipengaruhi jarak, ruangan, noise, dan speaker. Untuk hasil terbaik gunakan interface gitar dan headphone.
- USB Audio Interface harus kompatibel dengan dukungan USB Audio Android. Tidak semua interface, adaptor, atau ponsel mendukung kombinasi input/output yang sama.
- Aplikasi memproses satu sumber input aktif pada satu waktu pada versi awal. Penggabungan beberapa input dapat dipertimbangkan pada versi lanjutan.
- Latensi akhir dipengaruhi perangkat, driver, buffer, jalur USB, dan output. Angka latensi tidak boleh dijanjikan sebelum pengujian pada perangkat nyata.
- Tuner, metronom, perekam, looper, dan backing track dapat menjadi fitur tambahan setelah audio engine inti stabil.

3. Arsitektur sistem dan alur audio

Arsitektur disarankan memisahkan antarmuka Android dari mesin pemrosesan audio. Kotlin mengelola layar, izin, perangkat, dan preset. C++ menangani audio callback dan algoritma DSP agar jalur audio tetap ringan dan konsisten.

Lapisan	Tanggung jawab
UI — Kotlin / Jetpack Compose	Layar utama, pedalboard, knob/slider, status perangkat, preset, dan pengaturan.
Application layer — Kotlin	Izin, lifecycle, pemilihan input/output, komunikasi dengan native engine, dan penyimpanan preset.
Audio I/O — Oboe / AAudio	Membuka input/output stream, mengatur format dan buffer, menangani callback audio.
DSP engine — C++	Mengalirkan sampel audio melalui efek sesuai urutan, mengelola parameter, bypass, dan level.
Preset/data	Menyimpan nama preset, urutan efek, status aktif, dan nilai parameter.

Alur sinyal audio

1. Input: gitar melalui USB Audio Interface atau suara gitar yang ditangkap mikrofon HP.
2. Input conditioning: pemeriksaan level, format sampel, channel, dan pengendalian clipping.
3. DSP chain: Noise Gate → Compressor → Drive/Distortion → EQ → Modulation → Delay → Reverb.
4. Output conditioning: master level, limiter pengaman opsional, dan konversi format bila diperlukan.
5. Output: headphone, speaker, atau output interface yang tersedia.

Diagram ringkas

```
GUITAR → INPUT SELECTOR → AUDIO ENGINE → DSP EFFECT CHAIN → MASTER → AUDIO OUTPUT
```

Urutan efek sebaiknya dapat diubah pada versi lanjutan. Untuk versi pertama, gunakan urutan tetap yang mudah diuji dan menyediakan bypass individual.

4. Teknologi yang digunakan

Teknologi	Peran	Catatan
Kotlin	Bahasa utama aplikasi Android	Mengelola UI, lifecycle, izin, dan konfigurasi.
Jetpack Compose	Antarmuka pengguna	Cocok untuk kontrol interaktif dan tema gelap.
C++	Mesin DSP native	Memproses blok audio tanpa pekerjaan berat di UI thread.
Oboe	Audio I/O low-latency	Library Android native yang menggunakan AAudio bila tersedia dan jalur kompatibel.
CMake	Build native code	Mengompilasi library C++ melalui Android NDK.
DataStore	Penyimpanan preferensi/preset sederhana	Untuk preset kompleks dapat memakai JSON atau Room.

5. Struktur folder proyek Android Studio

Struktur berikut merupakan usulan awal. Nama folder dan file dapat disesuaikan selama tanggung jawab tiap modul tetap terpisah.

```
AltoGuitarFX/ ■■■ settings.gradle.kts ■■■ build.gradle.kts ■■■ gradle/ ■■■ app/ ■■■
build.gradle.kts ■■■ src/main/ ■■■ AndroidManifest.xml ■■■ java/com/altomedia/altoguitarfx/ ■
■■■ MainActivity.kt ■ ■■■ navigation/ ■ ■ ■■■ AppNavigation.kt ■ ■■■ ui/ ■ ■ ■■■ theme/ ■ ■
■■■ home/HomeScreen.kt ■ ■ ■■■ effects/EffectsScreen.kt ■ ■ ■■■ presets/PresetScreen.kt ■ ■
■■■ tuner/TunerScreen.kt ■ ■ ■■■ settings/SettingsScreen.kt ■ ■■■ audio/ ■ ■ ■■■
AudioEngine.kt ■ ■ ■■■ AudioDeviceManager.kt ■ ■ ■■■ AudioSettings.kt ■ ■■■ effects/ ■ ■ ■■■
EffectController.kt ■ ■■■ data/ ■ ■■■ Preset.kt ■ ■■■ PresetRepository.kt ■■■ cpp/ ■■■
CMakeLists.txt ■■■ native-lib.cpp ■■■ AudioEngine.h / AudioEngine.cpp ■■■ Effects.h /
Effects.cpp ■■■ NoiseGate.cpp ■■■ Compressor.cpp ■■■ Distortion.cpp ■■■ Equalizer.cpp ■■■
Chorus.cpp ■■■ Flanger.cpp ■■■ Phaser.cpp ■■■ Delay.cpp ■■■ Reverb.cpp
```

Tanggung jawab modul utama

- MainActivity.kt: titik masuk aplikasi dan integrasi tema/navigasi.
- AudioEngine.kt: API Kotlin untuk start/stop audio dan mengubah parameter engine native.
- AudioDeviceManager.kt: daftar perangkat input/output dan perubahan koneksi.
- EffectController.kt: mengirim status bypass dan nilai parameter efek.
- PresetRepository.kt: membaca/menyimpan preset.
- AudioEngine.cpp: callback audio, buffer, dan pemrosesan sinyal.
- Effects.cpp: pengelola rangkaian efek dan parameter DSP.

6. Rancangan antarmuka dan navigasi

Arah visual: tema gelap profesional, kontras tinggi, kontrol besar yang mudah disentuh, indikator status yang jelas, serta panel pedal yang menyerupai multi-effects hardware.

Layar	Isi utama
Home / Pedalboard	Input/output status, master volume, preset aktif, pedal efek, level meter, tombol start/stop.
Effects	Daftar efek, tombol ON/OFF, kontrol parameter, bypass, dan pengaturan urutan bila tersedia.
Presets	Preset bawaan, simpan sebagai preset baru, rename, duplikasi, hapus, dan restore default.
Tuner	Deteksi pitch, nama nada, indikator terlalu rendah/tinggi, dan referensi tuning.
Settings	Input device, output device, sample rate yang tersedia, buffer/performance, izin, dan informasi aplikasi.

Komponen layar Home

- Header ALTO GUITAR FX dan indikator status audio.
- Pilihan input: Internal Mic / USB Interface.
- Pilihan output yang tersedia.
- Input meter dan output meter untuk memantau sinyal.
- Pedalboard dengan status ON/OFF dan kontrol parameter.
- Preset aktif serta tombol simpan/muat preset.
- Master volume dan tombol mute/bypass keseluruhan.

Prinsip interaksi

- Setiap pedal memiliki status ON/OFF yang terlihat jelas.
- Perubahan knob/slider dikirim ke DSP engine dengan aman dan tanpa mengunci audio callback.
- Perubahan preset tidak boleh menimbulkan lonjakan volume; parameter penting sebaiknya dihaluskan (smoothing).
- Tampilkan pesan yang jelas bila tidak ada input, izin mikrofon ditolak, atau perangkat USB terputus.

7. Katalog efek dan parameter

Efek	Parameter utama	Fungsi
Noise Gate	Threshold, Release	Mengurangi noise saat sinyal gitar berada di bawah ambang.
Compressor	Threshold, Ratio, Attack, Release, Makeup	Mengendalikan dinamika dan sustain.
Overdrive	Drive, Tone, Level	Saturasi ringan hingga menengah.
Distortion	Gain, Tone, Level	Saturasi lebih kuat untuk rock.
Fuzz	Fuzz, Tone, Volume	Karakter saturasi tebal dan kasar.

Efek	Parameter utama	Fungsi
Equalizer	Bass, Mid, Treble; frekuensi/Q bila parametrik	Membentuk warna tonal.
Chorus	Rate, Depth, Mix	Modulasi pitch/waktu untuk kesan lebar.
Flanger	Rate, Depth, Feedback, Mix	Efek comb-filter bergerak.
Phaser	Rate, Depth, Feedback, Mix	Modulasi all-pass untuk karakter berputar.
Delay	Time, Feedback, Mix	Mengulang sinyal dengan jeda.
Reverb	Room/Size, Decay, Damping, Mix	Simulasi pantulan ruang.
Tremolo	Rate, Depth	Modulasi volume periodik.

Catatan implementasi DSP

Setiap efek perlu memiliki proses inisialisasi, pemrosesan blok sampel, pengaturan parameter, reset, dan bypass. Parameter yang berubah dari UI harus diterapkan melalui mekanisme thread-safe (misalnya atomic parameter atau command queue), bukan dengan mengakses objek DSP secara tidak aman dari dua thread.

8. Preset awal

Preset	Rangkaian awal yang disarankan	Karakter
CLEAN	Gate ringan → EQ → Reverb	Bersih dan jernih.
BLUES	Compressor ringan → Overdrive → EQ → Reverb	Hangat dan dinamis.
ACOUSTIC	EQ → Compressor ringan → Reverb	Menonjolkan karakter input; bukan pengganti pickup akustik.
CRUNCH	Gate → Overdrive → EQ	Crunch ringan.
ROCK	Gate → Distortion → EQ → Delay pendek	Rock ritmis.
HARD ROCK	Gate → Distortion → EQ → Chorus tipis	Lebih tebal.
METAL	Gate → Distortion → EQ → Compressor	Gain tinggi; perlu pengaturan noise.
LEAD SOLO	Overdrive → Delay → Reverb	Lead dengan sustain dan ruang.
AMBIENT	Chorus → Delay → Reverb	Ruang lebar dan atmosferik.

Nilai parameter preset harus diuji dengan gitar, pickup, level input, dan headphone nyata. Tabel ini hanya rancangan karakter awal, bukan setelan final yang dijamin cocok untuk semua gitar.

9. Spesifikasi audio dan performa

Aspek	Target/rancangan awal
Sample rate	Target 48 kHz; gunakan nilai yang didukung perangkat dan stream.
Format internal	Float 32-bit untuk pemrosesan DSP.
Channel	Input mono gitar sebagai dasar; output mono/stereo sesuai konfigurasi perangkat.
Buffer	Mulai dari buffer yang stabil, kemudian uji ukuran lebih kecil pada perangkat target.

Aspek	Target/rancangan awal
Threading	Audio callback berjalan pada jalur real-time; hindari disk I/O, logging berat, lock yang berpotensi blocking, dan alokasi memori.
Parameter	Gunakan smoothing/ramping untuk perubahan gain, mix, dan parameter agar tidak terdengar klik.
Clipping	Pantau level input/output dan sediakan pengendalian level untuk mengurangi clipping.
Latency	Ukur round-trip latency pada perangkat nyata; jangan menganggap angka teoretis sebagai hasil pengukuran.

Prinsip penting: audio real-time

Audio callback harus melakukan pekerjaan sesedikit mungkin dan memiliki waktu eksekusi yang dapat diprediksi. Siapkan buffer dan objek DSP sebelum callback berjalan. Hindari operasi yang dapat menunggu thread lain. Jika parameter UI berubah, kirim nilai melalui jalur komunikasi yang aman.

10. Izin Android, perangkat, dan kompatibilitas

- RECORD_AUDIO: diperlukan untuk merekam input mikrofon; minta izin runtime sebelum memulai capture.
- USB Host: deklarasikan fitur USB host bila aplikasi memerlukan koneksi perangkat USB; periksa dukungan perangkat dan permission USB saat perangkat terpasang.
- Audio routing: gunakan API audio Android/Oboe untuk mendeteksi dan memilih perangkat yang didukung. Ketersediaan port input dan output bisa berbeda.
- Lifecycle: hentikan atau pulihkan stream dengan benar saat aplikasi masuk background, panggilan masuk, perangkat dilepas, atau audio focus berubah.
- Headphone: sarankan headphone untuk mencegah feedback ketika memakai mikrofon internal.

11. Tahapan pembangunan per BAB

BAB	Pekerjaan	Hasil yang harus dicapai
1	Membuat proyek Android Studio, package, Gradle, tema dasar.	Aplikasi kosong dapat dibuild dan dijalankan.
2	Membuat navigasi dan kerangka layar.	Home, Effects, Presets, Tuner, Settings dapat dibuka.
3	Menyiapkan Android NDK, CMake, dan integrasi JNI.	Kotlin dapat memanggil fungsi native sederhana.
4	Mengintegrasikan Oboe dan membuka input/output stream.	Audio input/output dasar dapat diuji.
5	Memilih mikrofon internal dan mendeteksi USB.	Sumber input dapat dipilih dan status perangkat tampil.
6	Membangun audio callback dan level meter.	Sinyal lewat dengan stabil dan level dapat dipantau.
7	Membuat Noise Gate dan Compressor.	Efek dinamika berfungsi dan dapat di-bypass.
8	Membuat Overdrive, Distortion, dan Fuzz.	Efek gain/saturasi berfungsi.

BAB	Pekerjaan	Hasil yang harus dicapai
9	Membuat Equalizer.	Kontrol tonal bekerja.
10	Membuat Chorus, Flanger, Phaser, Tremolo.	Efek modulasi bekerja.
11	Membuat Delay dan Reverb.	Efek ruang/waktu bekerja.
12	Membuat pengelola urutan efek dan parameter.	Rangkaian efek dapat dikontrol dari UI.
13	Membuat sistem preset.	Preset dapat dimuat dan disimpan.
14	Membuat tuner.	Pitch dan tuning dapat dideteksi.
15	Pengaturan audio, routing, lifecycle, dan error handling.	Perangkat dan perubahan kondisi ditangani.
16	Optimasi performa dan uji perangkat.	Audio stabil, latensi terukur, crash diperbaiki.
17	Build release dan signing APK.	APK release siap diuji distribusi.

12. Rencana pengujian dan kriteria selesai

Pengujian perangkat

- Uji minimal satu ponsel Android dengan mikrofon internal.
- Uji USB Audio Interface yang diketahui kompatibel, termasuk proses izin USB.
- Uji headphone kabel dan output interface bila didukung.
- Uji saat kabel USB dicabut dan dipasang kembali.
- Uji saat aplikasi berpindah background/foreground dan saat audio focus berubah.

Pengujian audio

- Pastikan input dapat terdengar pada output dengan jalur monitoring yang disengaja.
- Uji setiap efek satu per satu, bypass, dan kombinasi beberapa efek.
- Uji perubahan parameter saat audio berjalan untuk mendeteksi bunyi klik/pop.
- Uji level rendah dan tinggi untuk melihat noise, clipping, dan distorsi yang tidak diinginkan.
- Ukur round-trip latency menggunakan metode pengukuran yang konsisten pada perangkat nyata.
- Jalankan uji stabilitas jangka panjang untuk mendeteksi drop-out, crackle, dan crash.

Kriteria versi awal dianggap selesai

- Aplikasi dapat memilih input mikrofon atau USB yang kompatibel.
- Audio dapat diproses dan didengar secara real-time melalui output yang didukung.
- Minimal Noise Gate, Overdrive, Distortion, EQ, Delay, dan Reverb berfungsi.
- Kontrol UI benar-benar mengubah hasil suara.
- Preset dapat disimpan dan dimuat tanpa merusak konfigurasi.
- Penanganan izin, perangkat terputus, dan error audio berjalan dengan jelas.

13. Risiko teknis dan mitigasi

Risiko	Dampak	Mitigasi
USB interface tidak kompatibel	Input/output tidak terdeteksi atau hanya salah satu arah bekerja.	Uji perangkat tertentu; tampilkan daftar perangkat dan pesan error yang informatif.
Latensi terlalu tinggi	Permainan terasa terlambat.	Gunakan Oboe, jalur low-latency, buffer sesuai perangkat, dan ukur round-trip latency.
Audio crackle/drop-out	Suara putus atau pecah.	Optimalkan callback, kurangi beban DSP, hindari lock/alokasi, uji CPU dan buffer.
Feedback mikrofon	Suara melengking atau berulang.	Anjurkan headphone, atur gain, dan beri peringatan saat memakai speaker.
Perubahan parameter menimbulkan klik	Suara tidak halus.	Gunakan parameter smoothing/ramping.
Perbedaan perangkat Android	Perilaku audio tidak konsisten.	Uji pada beberapa model dan dokumentasikan perangkat yang tervalidasi.

14. Checklist sebelum rilis APK

- Nama aplikasi, ikon, versi, dan package name sudah final.
- Izin mikrofon dan USB dijelaskan kepada pengguna.
- Tidak ada crash pada alur start/stop audio dan pergantian perangkat.
- Semua tombol dan knob yang ditampilkan memiliki fungsi nyata.
- Audio callback tidak melakukan operasi blocking atau alokasi tak terduga.
- Preset default dan penyimpanan preset sudah diuji.
- Build release ditandatangani dengan keystore yang disimpan aman.
- Uji instalasi APK release pada perangkat Android target.
- Sediakan informasi kompatibilitas interface audio dan rekomendasi headphone.

Lampiran A — keputusan awal proyek

Item	Keputusan rancangan
Nama aplikasi	ALTO GUITAR FX
Package name	com.altomedia.altoguitarfx
Sumber input	Mikrofon internal HP dan USB Audio Interface; satu input aktif pada satu waktu di versi awal.
Mode	Real-time.
Kategori	Professional Multi Effects.
Bahasa	Kotlin + C++.
UI	Jetpack Compose, tema gelap profesional.
Audio engine	Oboe + Android NDK/CMake.
Target sample rate	48 kHz bila didukung perangkat.

Lampiran B — referensi teknis resmi

Gunakan dokumentasi resmi berikut saat implementasi karena detail API, versi Android, dan library dapat berubah:

- Android NDK: https://developer.android.com/ndk
- Oboe audio library: https://github.com/google/oboe
- Android audio architecture: https://developer.android.com/media/platform/audio
- Jetpack Compose: https://developer.android.com/compose
- Android USB host: https://developer.android.com/develop/connectivity/usb/host

---

Penutup
Dokumen ini menjadi blueprint pengembangan ALTO GUITAR FX. Tahap berikutnya adalah membuat project Android Studio yang dapat dibuild, kemudian mengintegrasikan Oboe dan menguji jalur audio dasar sebelum menambahkan efek DSP satu per satu.

Catatan: dokumen ini adalah rancangan teknis lengkap untuk memulai proyek, bukan source code lengkap atau APK yang sudah berfungsi.